



Initiation à l'architecture orientée services

django

REST
framework

UP Web

AU: 2025 / 2026

Plan



1.Introduction : Pourquoi les services web ?

2. Les familles de Services Web

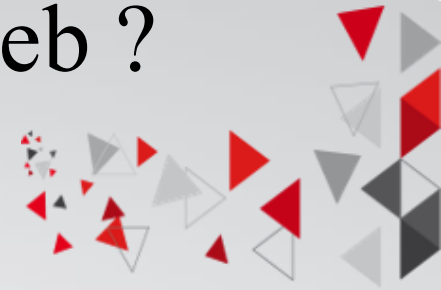
3. Définition de SOA (Service-Oriented Architecture)

4.Exemple d'un flux d'une application RESTFUL

5.Django Rest Framework

Introduction : Pourquoi les services web ?

Paradigme logiciel

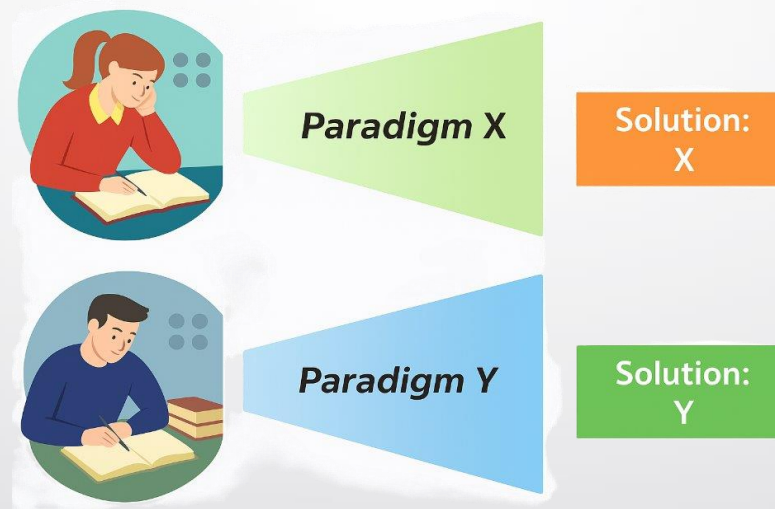


Le terme de paradigme est employé pour exprimer la façon dont un système a été conçu et pensé dans ses grandes lignes.



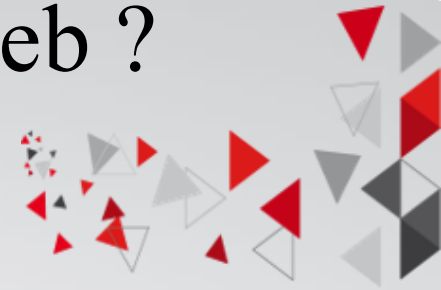
Objectif:

Développer une application de facturation



Introduction : Pourquoi les services web ?

Paradigme logiciel



- Les révolutions informatiques coïncident généralement avec un changement de paradigme.
- Niveau d'abstraction grandissant avec l'évolution des paradigmes
- On distingue l'évolution des paradigmes logicielles comme suit:



Introduction : Pourquoi les services web ?

Paradigme logiciel



- **Un service** est un terme général utilisé en informatique pour décrire une unité autonome de fonctionnalités qui peut être exécutée de manière indépendante.
- Il possède une interface pour la communication et l'interaction avec un demandeur de service.
- Service web = **service** + **web**
- Un service Web met à disposition un service **via Internet**
- Un SW est un **programme informatique**, permettant la communication et l'échange de données entre applications et systèmes **hétérogènes** dans des environnements **distribués**.
- Les services Web interagissent à travers l'échange de messages.

Les familles de Services Web



Il existe deux grandes familles de services web:

- Les services web **étendus** utilisant les standards:
 - SOAP (Simple Object Access Protocole) pour la communication
 - UDDI (Universal Description, Discovery, and Integration) (annuaire) pour la publication
 - WSDL (web Service Description Language) (contrat) pour la descriptionFormat de données échangées : Seulement le XML.
- Les services web **REST** (Representational State Transfer) utilisant:
 - Directement HTTP au lieu d'une enveloppe SOAP
 - Un URI pour nommer et identifier une ressource
 - Les méthodes HTTP (POST, GET, PUT et DELETE) pour effectuer les opérations de base CRUD

Plusieurs formats de données supportés : XML, JSON,HTML,TXT

Les familles de Services Web



Critère	SOAP	REST
Protocole	Utilise SOAP – enveloppe XML sur HTTP	Utilise directement HTTP
Format de données	XML uniquement	JSON, XML, HTML, TXT
Description	Basé sur WSDL (contrat formel)	Utilise URI pour identifier les ressources
Complexité	Plus complexe à configurer	Plus simple et léger
Performance	Moins performant – verbeux	Rapide et léger
Utilisation typique	Application d'entreprise	API Web, Microservices, mobiles

Définition de la SOA (Service-Oriented Architecture)



- “ L’architecture orientée service constitue un **style d’architecture** basée sur le principe de séparation de l’activité métier en une série de **services**”.
- “ Ces services peuvent **être assemblés et liés** entre eux selon le principe de couplage lâche pour exécuter l’application désirée. ”

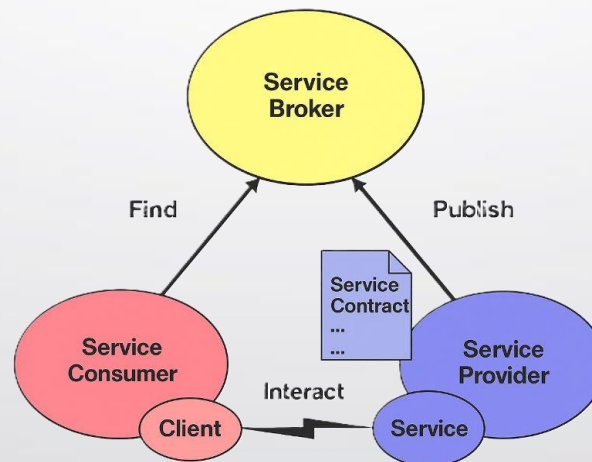
Gartner - Septembre 2005

Définition de la SOA (Service-Oriented Architecture)



Eléments SOA

- **Le fournisseur de service** crée le service Web, puis publie son interface ainsi que les informations d'accès au service, dans un annuaire de services Web.
- **L'annuaire de service** rend disponible l'interface du service ainsi que ses informations d'accès, pour n'importe quel demandeur potentiel de service (peut être public ou privé).
- **Le consommateur de service** accède à l'annuaire de service pour effectuer une recherche afin de trouver les services désirés. Ensuite, il se lie au fournisseur pour invoquer le service.



Exemple d'un flux d'une application RESTFUL



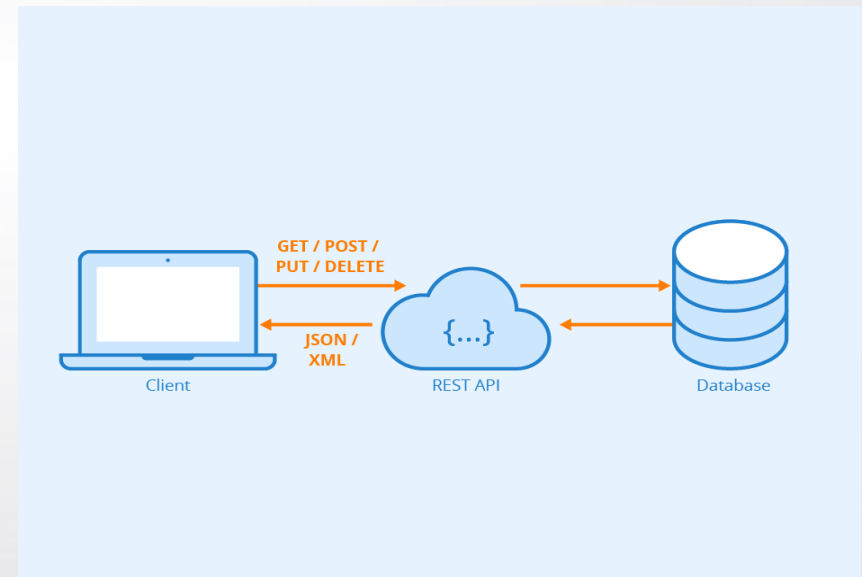
Principes de REST

Une ressource

- Un identifiant de ressource
- Une représentation de la ressource
- Interagir avec les ressources avec les requêtes HTTP : GET, POST, PUT et DELETE

CRUD

GET	→	Récupérer la représentation de la ressource
POST	→	Créer une nouvelle ressource
PUT	→	Modifier partiellement une ressource
DELETE	→	Supprimer une ressource



Exemple d'un flux d'une application RESTFUL



Ressource (Identifiant): Identifié par une URI

Exemple : `http://localhost:8080/api/devices`

Méthodes (Verbes)

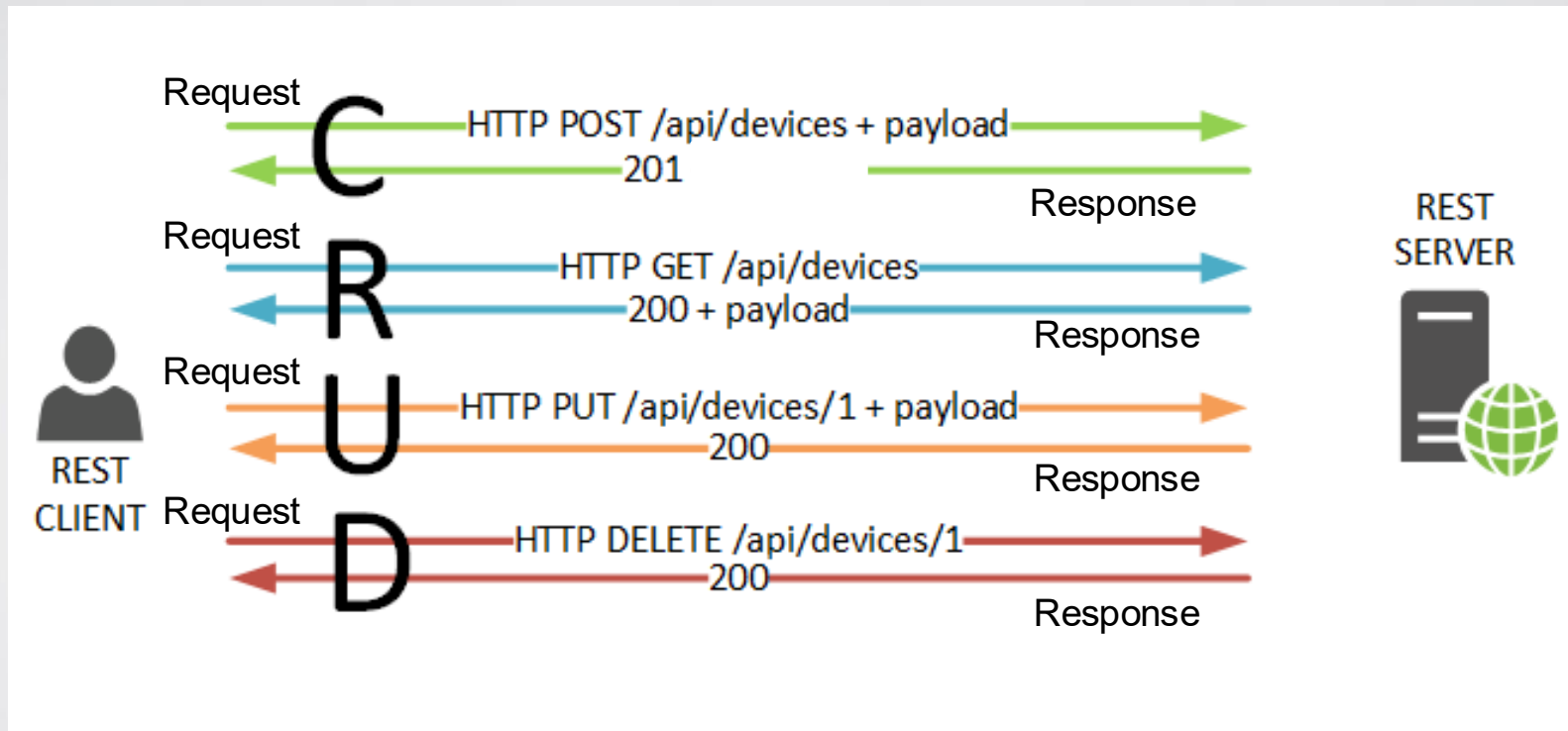
- Pour manipuler la ressource
- Méthodes HTTP : GET, POST, PUT and DELETE

Représentation

- Donne une vue sur l'état de la ressource
- Informations transférées entre le client et le serveur

Exemples de format de données : XML, Text, JSON, ...

Exemple d'un flux d'une application RESTFUL



Django Rest Framework



Django REST Framework est une boîte à outils puissante et flexible pour la création d'API Web (Service web).

Voici quelques raisons pour lesquelles vous pourriez vouloir utiliser Django REST Framework :

Interface web navigable

- est un énorme avantage en termes d'ergonomie pour les développeurs.

Un système de sérialisation

- qui prend en charge à la fois les sources de données basées sur un ORM (comme les modèles Django) et celles qui ne le sont pas.

Une personnalisation totale

- jusqu'au plus bas niveau — vous pouvez simplement utiliser des vues basées sur des fonctions si vous n'avez pas besoin des fonctionnalités avancées.

Une documentation complète

- une excellente communauté de support.

Reconnu internationalement

- Utilisé et approuvé par des entreprises telles que **Mozilla, Red Hat, Heroku et Eventbrite.**

<https://www.django-rest-framework.org/>

Django Rest Framework



Le Django REST Framework nécessite les éléments suivants :

- **Django** : versions 4.2, 5.0, 5.1 ou 5.2
- **Python** : versions 3.10, 3.11, 3.12, 3.13 ou 3.14

Installation et configuration

- Installation

```
pip install djangorestframework
```

ou bien

```
uv pip install django djangorestframework
```

- Ajout dans settings.py

```
INSTALLED_APPS = [  
    'rest_framework',  
    'myapp', # ton application  
]
```

Django Rest Framework



Sérialisation (Serializers)

Les serializers convertissent les objets Django (Model) en **JSON**.

```
from rest_framework import serializers
from .models import Article

class ArticleSerializer(serializers.ModelSerializer):
    class Meta:
        model = Article
        fields = '__all__'
```

Django Rest Framework



Vues API (Views)

- **APIView (base orientée classe)**

sont les composants qui définissent la logique de traitement des requêtes HTTP (GET, POST, PUT, DELETE, etc.).

Elles sont similaires aux vues Django classiques, mais elles renvoient généralement des réponses au format JSON, adaptées aux clients d'API (applications web, mobiles, etc.).

```
from rest_framework.views import APIView
from rest_framework.response import Response
from .models import Article
from .serializers import ArticleSerializer

class ArticleList(APIView):
    def get(self, request):
        articles = Article.objects.all()
        serializer = ArticleSerializer(articles, many=True)
        return Response(serializer.data)
```


Django Rest Framework



- **ViewSet:**

est une classe de base fournie par Django REST Framework qui permet de créer des vues orientées objet pour les API.

Elle fonctionne comme les class-based views (CBV) de Django, mais elle ajoute des fonctionnalités propres aux API REST, comme :

- la gestion automatique du format de réponse (JSON, XML, etc.),
- la validation des données,
- la gestion des permissions et de l'authentification,
- la génération d'une interface web interactive pour tester les endpoints.

```
from rest_framework import viewsets

class ArticleViewSet(viewsets.ModelViewSet):
    queryset = Article.objects.all()
    serializer_class = ArticleSerializer
```

Django Rest Framework



- **ViewSets - Filtrage et Recherche :**

Pour activer ces fonctionnalités dans ViewSet, il faut installer **django-filter** et le configurer comme suit :

```
#Dans settings.py
INSTALLED_APPS = [
...
'rest_framework',
'django_filters',
....
]
REST_FRAMEWORK = {
'DEFAULT_FILTER_BACKENDS': [
'django_filters.rest_framework.DjangoFilterBackend',
'rest_framework.filters.SearchFilter',
'rest_framework.filters.OrderingFilter',
]
}
```

Django Rest Framework



- **ViewSets - Filtrage et Recherche :**

1. Filtrage précis (filterset_fields)

Ce filtre permet d'obtenir uniquement les objets qui répondent à des critères exactement égaux aux valeurs données dans l'URL.

2. Recherche textuelle (search_fields)

Ce filtre permet de faire une recherche partielle (par sous-chaîne) sur un ou plusieurs champs textuels.

Il s'utilise avec le paramètre search dans l'URL.

3. Tri dynamique (ordering_fields)

Permet de trier les résultats selon un ou plusieurs champs spécifiés.

Django Rest Framework



- **ViewSets - Filtrage et Recherche :**

Pour activer ces fonctionnalités dans ViewSet, il faut installer **django-filter** et le configurer comme suit :

```
#Dans la classe au niveau de views.py
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework import viewsets ,filters

class ....(viewsets.ModelViewSet):
    ....
    filter_backends = [DjangoFilterBackend,
filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = [.....]
    search_fields = [...]
    ordering_fields = [.....]
    ordering = [.....]
```



Atelier